

Computer Architecture

Course No. 21472

Week #3

Instructions: Language of the Computer

Bumsoo Kim

School of Computer Science & Engineering

E-mail : bumsoo@cau.ac.kr

Office : 310-430

Retrospect: RISC vs. CISC

RISC (Reduced Instruction Set Computing) architecture

- Example : MIPS architecture, ARM, PowerPC, etc.
- MIPS: **M**icroprocessor without **I**nterlocked **P**ipeline **S**tages, refers to a RISC architecture in the early 1980s
- Advantages: Simplified and Small Set of Instructions
  - Faster Execution
  - Power-efficient
  - Scalability: Easier implementation of features like parallelism
- Disadvantages: Simplified and Small Set of Instructions
  - More Instructions for Complex Tasks: increasing program size

CISC (Complex Instruction Set Computing) architecture

- Example : x86, x86-64 (Intel, AMD)
- Minimize the number of instructions per program by providing large and complex set of instructions
- Advantages
  - Reduced Program Size
  - Compatibility with older instructions
- Disadvantages
  - Slower Execution and Higher Power Consumption

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

MIPS Instruction Set

- Stanford MIPS commercialized by MIPS technologies (www.mips.com)
- Large share of embedded core market
- Typical of many modern ISAs

Classification

Classification by types

- Arithmetic Instructions
- Memory (Data Transfer) Instructions
- Logical Instructions
- Conditional Instructions
- Branch/Jump Instructions
- Floating Point Instructions
- Pseudo Instructions

Classification by formats

- R-format
- I-format
- J-format
- Floating Point Instructions format

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

MIPS Instruction Set

- Stanford MIPS commercialized by MIPS technologies (www.mips.com)
- Large share of embedded core market
- Typical of many modern ISAs

Classification

Classification by types

- Arithmetic Instructions
- Memory (Data Transfer) Instructions
- Logical Instructions
- Conditional Instructions
- Branch/Jump Instructions
- Floating Point Instructions
- Pseudo Instructions

Classification by formats

- R-format
- I-format
- J-format
- Floating Point Instructions format

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Arithmetic Operation

Format

- One destination + Two sources

```
add a, b, c     # a = b + c
```
: All arithmetic operations follow this form

Design Principle 1 : **Simplicity** favors **Regularity**

- Regularity makes implementation simpler
- Simplicity enables higher performance at lower cost

Example

Given the C code below, write the compiled MIPS code.

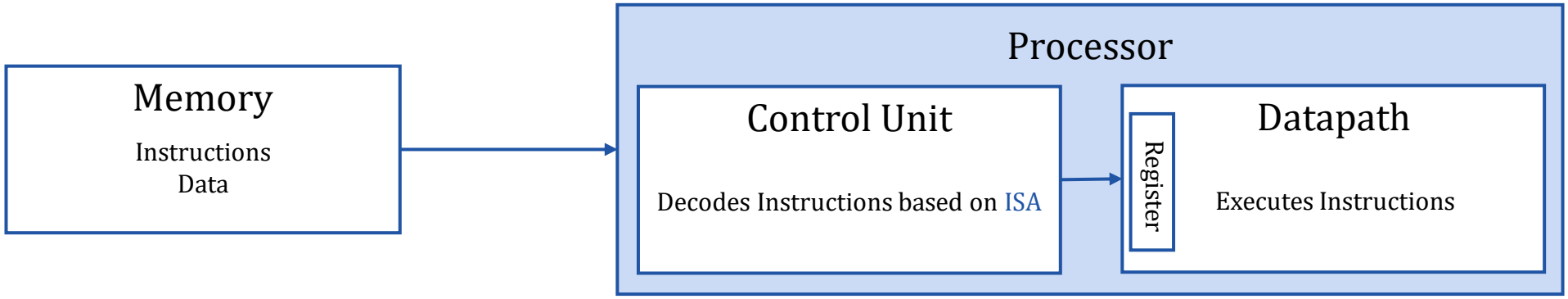
1. a = b + c + d;
2. f = (g + h) – (i + j);

A.

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Register Operands

- Arithmetic Instructions use "register" operands
- Registers are special locations built directly in processor
- MIPS has 32 × '32-bit registers' (numbered 0~31)



Register : Fastest memory in computer system located directly within the processor.

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Register Operands

- Arithmetic Instructions use "register" operands
- Registers are special locations built directly in processor
- MIPS has 32 × '32-bit registers' (numbered 0~31)

Q. Why '32-bit registers' ?

- MIPS was designed when 32 bit architecture were the standard for general-purpose computing
- "Word size" : In 32-bit architectures, processor can handle data and instructions in 32-bit chunks
- Even with later versions like MIPS64, 32-bit registers remained significant for compatibility with legacy systems and software

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Register Operands

- Arithmetic Instructions use "register" operands
- Registers are special locations built directly in processor
- MIPS has 32 × '32-bit registers' (numbered 0~31)
- Assembler names
  - Temporary Registers (x10) : `$t0, $t1,…,$t9`
  - Saved Registers (x8) : `$s0, $s1,…,$s7`

Design Principle 2 : **Smaller** is **Faster**

- We design very small registers for faster operations on frequently accessed data

Example

Given the C code below, write the compiled MIPS code (access variables via register operands).

```
f = ( g + h ) – ( i + j );      # f, … , j stored in $s0, …, $s4
```

A.

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

MIPS Instruction Set

- Stanford MIPS commercialized by MIPS technologies (www.mips.com)
- Large share of embedded core market
- Typical of many modern ISAs

Classification

Classification by types

- Arithmetic Instructions
- Memory (Data Transfer) Instructions
- Logical Instructions
- Conditional Instructions
- Branch/Jump Instructions
- Floating Point Instructions
- Pseudo Instructions

Classification by formats

- R-format
- I-format
- J-format
- Floating Point Instructions format

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Memory Operands

Main Memory

- Used for composite data
- Arrays, Structures, Dynamic Data
- Not all variables can be stored in registers (since registers are small, 32-bit)

Arithmetic Operations require Data Transfer Instructions

- Load values from memory into registers
- Store result from register to memory

To access memory, we need memory address

- Memory is byte addressed : Each address identifies an 8-bit byte
- Words (32-bit) are *aligned* in memory : Address must be a multiple of 4

Endian

- In 'Big Endian', the Most Significant Byte (MSB) is stored at the lowest memory address.
- In 'Little Endian', the Least Significant Byte (LSB) is stored at the lowest memory address.

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Memory Operands

Example

Given a 32-bit (4-byte) integer value in hexadecimal 0x12345678 and the memory address 0-3, compare the values stored in each address in ‘Big Endian’ vs. ‘Little Endian’.

A.

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Memory Operands

Example

Given a 32-bit (4-byte) integer value in hexadecimal 0x34125678 and the memory address 0-3, compare the values stored in each address in ‘Big Endian’ vs. ‘Little Endian’.

A.

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Memory Operands

- load word (lw) and store word (sw)
  - Three operands : destination register, offset, base address

```
lw dest , offset ( base address )
```

Example

Given the C code

```
g = h + A[8];   # A is an array, g in $s1, h in $s2 , the base address of A in $s3
```

A.

Example

Given the C code

```
A[12] = h + A[8];   # h in $s2, base address of A in $s3
```

A.

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Registers vs. Memory

- Registers are faster to access than memory
  - More simpler to use
- Operating on memory data requires loads and stores
  - Arithmetic instructions work on registers only
  - More instructions to be executed
- Compiler must use registers for variables as much as possible
  - Only spill to memory for less frequently used variables
  - Register optimization is important for efficiency

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Immediate Operands

- Constant data specified in an instruction

```
addi $s3, $s3, 4
```

  - There is no immediate instruction for subtract. Use negative constant for addi

```
addi $s2, $s1, -1
```

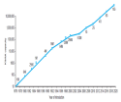
- MIPS register 0 ($zero) is the constant 0
  - Cannot be overwritten
  - Useful for common operations (e.g., move between registers : `add $t2, $s1, $zero`)

Design Principle 3 : **Make the common case fast**

- Small constants are common
- Immediate operand avoids load instruction

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Retrospect: Eight Great Ideas in Computer Architecture



Design for *Moore's Law*



Use *abstraction* to simplify design



Make the *common case fast*



Hierarchy of memories



Dependability via redundancy



Performance *via parallelism*



Performance *via pipelining*



Performance *via prediction*

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Unsigned Binary Integers

- Given an n-bit number

$$x = x_{n-1}2^{n-1} + x_{n-2}2^{n-2} + \cdots + x_1 2^1 + x_0 2^0$$

- Range: 0 to $+2^n - 1$

32-bit

$1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111_2 = 4{,}294{,}967{,}295_{10}$

$0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 1011_2$

$= 0 + \ldots + 1\times2^3 + 0\times2^2 + 1\times2^1 + 1\times2^0$

$= 0 + \ldots + 8 + 0 + 2 + 1 = 11_{10}$

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Unsigned Binary Integers

Then, how should we represent negative numbers?

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

2s-Complement Signed Integers

- Given an n-bit number

$$x = -x_{n-1}2^{n-1} + x_{n-2}2^{n-2} + \cdots + x_1 2^1 + x_0 2^0$$

- Bit 31 represents sign bit
  - '1' for negative numbers
  - '0' for non-negative numbers
  - Non-negative numbers have the same unsigned and 2s-complement representation

- Range: -2^{n-1} to $+2^{n-1} - 1$

$1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111_2$
$= -1\times2^{31} + 1\times2^{30} \ldots + 1\times2^3 + 1\times2^2 + 1\times2^1 + 1\times2^0 = -2{,}147{,}483{,}648 + 2{,}147{,}483{,}647 = -1$

$1000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000_2$
$= -1\times2^{31} = -2{,}147{,}483{,}648$

$0111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111_2$

$= 0\times2^{31} + 1\times2^{30} \ldots + 1\times2^3 + 1\times2^2 + 1\times2^1 + 1\times2^0 = \frac{2^{31}-1}{2-1} = 2{,}147{,}483{,}648 - 1 = 2{,}147{,}483{,}647$

$S_n = a + ar + ar^2 + \cdots + ar^{n-1}$ $\quad r \neq 1$이면 $S_n = \frac{a(1-r^n)}{1-r}$

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Signed Negation

- Signed Negation: "Complement and add 1"

$$x + \overline{x} = 1111\ldots111_2 = -1$$

$$\overline{x} + 1 = -x$$

- Example: negate +2

"+2" = 0000 0000 … 0000 0010_2
"-2" = 1111 1111 … 1111 1101_2+1_2=1111 1111 … 1111 1110_2

Example

Negate +10

A.

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Signed Extension

- Representing a number using more bits
  - Preserve the numeric value
- In MIPS instruction set
  - `addi` : extend immediate value
  - `lb, lh` : extend loaded byte/halfword
  - `beq, bne` : extend the displacement
- Replicate the sign bit to the left
- Examples: 8-bit to 16-bit

  “+2” = 0000 0010 => 0000 0000 0000 0010
  “-2” = 1111 1110 => 1111 1111 1000 0010

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Representing Instructions

- Instructions are encoded in binary
  - Called machine code
- MIPS instruction
  - Encoded as “32-bit” instruction words
  - Small number of instruction formats encoding operation, registers, etc.
  - Regularity!
- Register numbers
  - $t0-$t7 : reg’s 8-15
  - $t8-$t9 : reg’s 24-25
  - $s0-s7 : reg’s 16-23

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Hexadecimal

Base 16

- Compact representation of bit strings
- 4 bits per hex digit

0	0000	4	0100	8	1000	c	1100
1	0001	5	0101	9	1001	d	1101
2	0010	6	0110	a	1010	e	1110
3	0011	7	0111	b	1011	f	1111

Example

Convert “eca8 6420” to binary code

A.

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

MIPS Instruction Set

- Stanford MIPS commercialized by MIPS technologies (www.mips.com)
- Large share of embedded core market
- Typical of many modern ISAs

Classification

Classification by types

- Arithmetic Instructions
- Memory (Data Transfer) Instructions
- Logical Instructions
- Conditional Instructions
- Branch/Jump Instructions
- Floating Point Instructions
- Pseudo Instructions

Classification by formats

- R-format
- I-format
- J-format
- Floating Point Instructions format

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

MIPS R-format Instructions

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

- Instruction fields
  - **op** : operation code (opcode) – always 0 in R-format because the operation is determined by 'funct'.
  - **rs** : first source register number
  - **rt** : second source register number
  - **rd** : destination register number
  - **shamt** : shift amount (00000 for now)
  - **funct** : function code (extends opcode)

- Example

```
add $t0, $s1, s2
```

000000	$s1	$s2	$t0	0	add
0	17	18	8	0	32
000000	10001	10010	01000	00000	100000

$00000010001100100100000000100000_2 = 02324020_{16}$

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

MIPS Instruction Set

- Stanford MIPS commercialized by MIPS technologies (www.mips.com)
- Large share of embedded core market
- Typical of many modern ISAs

Classification

Classification by types

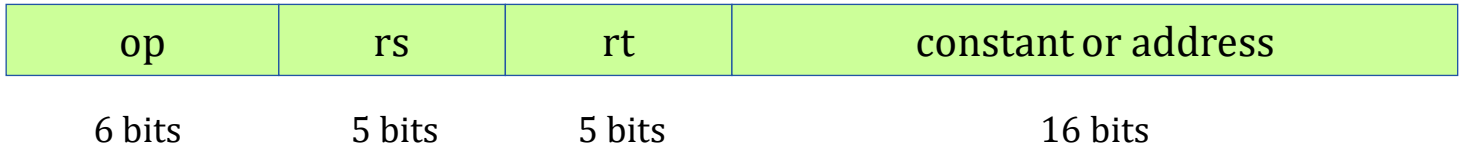
- Arithmetic Instructions
- Memory (Data Transfer) Instructions
- Logical Instructions
- Conditional Instructions
- Branch/Jump Instructions
- Floating Point Instructions
- Pseudo Instructions

Classification by formats

- R-format
- I-format
- J-format
- Floating Point Instructions format

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

MIPS I-format Instructions



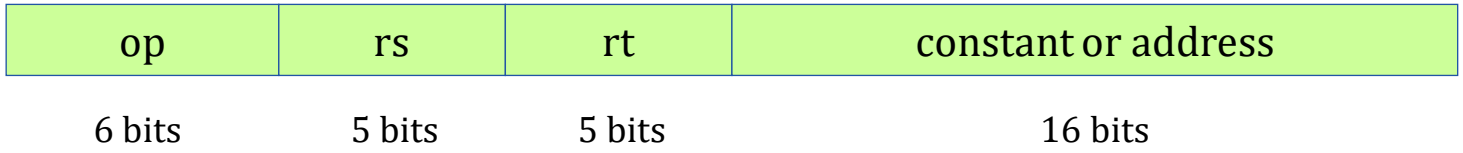
- Immediate arithmetic and load/store instructions
  - **rt** : destination or source register number
  - **constant** : -2^{15} to $+2^{15}-1$
  - **address** : offset added to base address in **rs** (e.g., lw, sw)
- Examples : `addi, lw, sw, beq, bne, slti, andi, ori, lui, …`

Design Principle 4 : **Good design demands good compromises**

- Different formats complicate decoding, but allow 32-bit instructions uniformly
- Keep formats as similar as possible

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

MIPS I-format Instructions



Example

C code : A[300] = h + A[300]

A.

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

MIPS Instruction Set

- Stanford MIPS commercialized by MIPS technologies (www.mips.com)
- Large share of embedded core market
- Typical of many modern ISAs

Classification

Classification by types

- Arithmetic Instructions
- Memory (Data Transfer) Instructions
- Logical Instructions
- Conditional Instructions
- Branch/Jump Instructions
- Floating Point Instructions
- Pseudo Instructions

Classification by formats

- R-format
- I-format
- J-format
- Floating Point Instructions format

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Logical Operations

- Instructions for bitwise manipulation

Operation	C	Java	MIPS
Shift left	<<	<<	`sll`
Shift right	>>	>>>	`srl`
Bitwise AND	&	&	`and, andi`
Bitwise OR	|	|	`or, ori`
Bitwise NOT	~	~	`nor`

- Useful for extracting and inserting groups of bits in a word

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Logical Operations

Shift Operations

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

- shamt : how many positions to shift
- Shift Left Logical (SLL)
  - Shift left and fill with 0 bits
  - `sll` by i -bits multiplies by 2^i
- Shift Right Logical (SRL)
  - Shift right and fill with 0 bits
  - `srl` by i -bits divides by 2^i (unsigned only)

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Logical Operations

AND/OR Operations

- AND : Useful to mask bits in a word
  - Select bits and clear others to 0
  - Example : `and $t0, $t1, $t2`

$t2	0000 0000 0000 0000 0000 1101 1100 0000
$t1	0000 0000 0000 0000 0011 1100 0000 0000
$t0	0000 0000 0000 0000 0000 1100 0000 0000

- OR : Useful to include bits in a word
  - Select bits to 1, leave others unchanged
  - Example : `or $t0, $t1, $t2`

$t2	0000 0000 0000 0000 0000 1101 1100 0000
$t1	0000 0000 0000 0000 0011 1100 0000 0000
$t0	0000 0000 0000 0000 0011 1101 1100 0000

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Logical Operations

NOT Operations

- Useful to invert bits in a word
  - Change 0 to 1, and 1 to 0

- MIPS has NOR 3-operand instruction

```
nor $t0, $t1, $zero
```

$t1	0000	0000	0000	0000	0011	1100	0000	0000
$t0	1111	1111	1111	1111	1100	0011	1111	1111

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

MIPS Instruction Set

- Stanford MIPS commercialized by MIPS technologies (www.mips.com)
- Large share of embedded core market
- Typical of many modern ISAs

Classification

Classification by types

- Arithmetic Instructions
- Memory (Data Transfer) Instructions
- Logical Instructions
- Conditional Instructions
- Branch/Jump Instructions
- Floating Point Instructions
- Pseudo Instructions

Classification by formats

- R-format
- I-format
- J-format
- Floating Point Instructions format

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Conditional Operations

- Branch to a labeled instruction if a condition is true, otherwise continue sequentially

- `beq rs, rt, L1`
  - if (rs == rt) branch to instruction labeled L1
- `bne rs, rt, L1`
  - if (rs != rt) branch to instruction labeled L1
- `j L1`
  - Unconditional jump to instruction labeled L1

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Conditional Operations

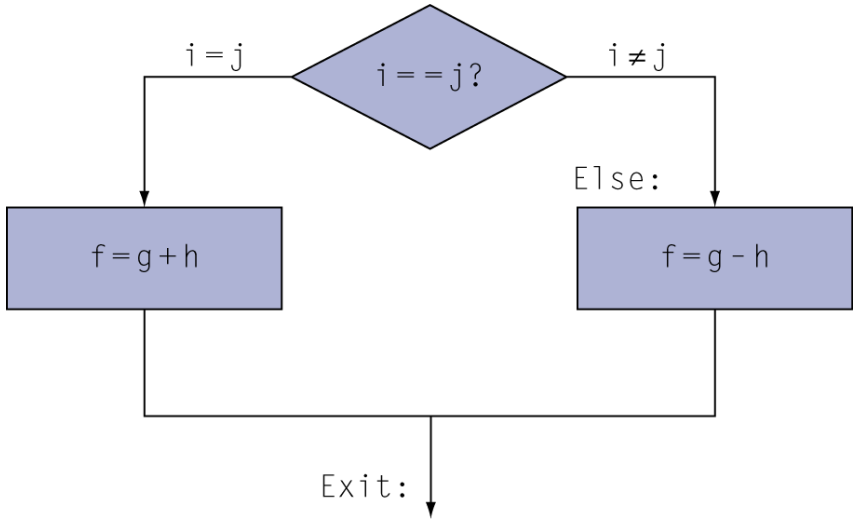
Compiling If Statements

- Given a C code :

```
if ( i == j ) f = g + h;
else f = g – h;
```

- f, g, … in $s0, $s1, …

A.



Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Conditional Operations

Compiling Loop Statements

- Given a C code :

  While (save[i] == k) i += 1

  - i in $s3, k in $s5, address in $s6

A.

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Conditional Operations

More Operations

- `slt rd, rs, rt`
  - set-on-less-than : if (rs < rt) rd=1; else rd=0;
- `slti rt, rs, constant`
  - set-on-less-than-immediate : if (rs < constant) rt=1; else rt=0;
- Use in combination with `beq, bne`
  - To create all relative conditions { ==, !=, <, <=, >, >= }
  - Example :

```
slt $t0, $s1, $s2   # if $s1 < $s2, then 1 will be stored at $t0
bne $t0, $zero, L   # if $s1 < $s2, since 1 != 0, branch to L
```

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Conditional Operations

Branch Instruction Design

- Why not just use `blt, bge`
  - branch-on-less-than, branch-on-greater-than-or-equal
  - Hardware <, > slower than =, !=
    - Combining with branch involves more work per instruction, requiring a slower clock
    - All instructions penalized!
  - Good design compromise : `beq, bne`

```
slt $t0, $s1, $s2
bne $t0, $zero, L
```

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Conditional Operations

Signed vs. Unsigned

- Signed comparison: `slt, slti`
- Unsigned comparison: `sltu, sltui`

- Example

$s0 = 1111 1111 1111 1111 1111 1111 1111 1111
$s1 = 0000 0000 0000 0000 0000 0000 0000 0001

```
slt $t0, $s0, $s1   # signed
```

- Since -1 < +1, $t0 = 1

```
sltu $t0, $s0, $s1   # unsigned
```

- Since +4,294,967,295 > +1, $t0 = 0

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

MIPS Instruction Set

- Stanford MIPS commercialized by MIPS technologies (www.mips.com)
- Large share of embedded core market
- Typical of many modern ISAs

Classification

Classification by types

- Arithmetic Instructions
- Memory (Data Transfer) Instructions
- Logical Instructions
- Conditional Instructions
- Branch/Jump Instructions
- Floating Point Instructions
- Pseudo Instructions

Classification by formats

- R-format
- I-format
- J-format
- Floating Point Instructions format

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Procedure Calling

Steps Required

1. Place parameters in Registers
2. Transfer control to procedure
3. Acquire Storage for procedure
4. Perform Procedure's operations
5. Place result in Register for caller
6. Return to place of call

PC : program counter

Register containing the address of the instruction in the program being executed

$ra : return address

Register containing the return address of a procedure

Procedure Call

```
jal ProcedureLabel
```

- jump-and-link
- Address of following instruction (**PC+4**) put in **$ra**
- Jumps to target address

Procedure Return

```
jr $ra
```

- jump-register
- Copies **$ra** to program counter (PC)
- Can also be used for computed jumps

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Register Usage

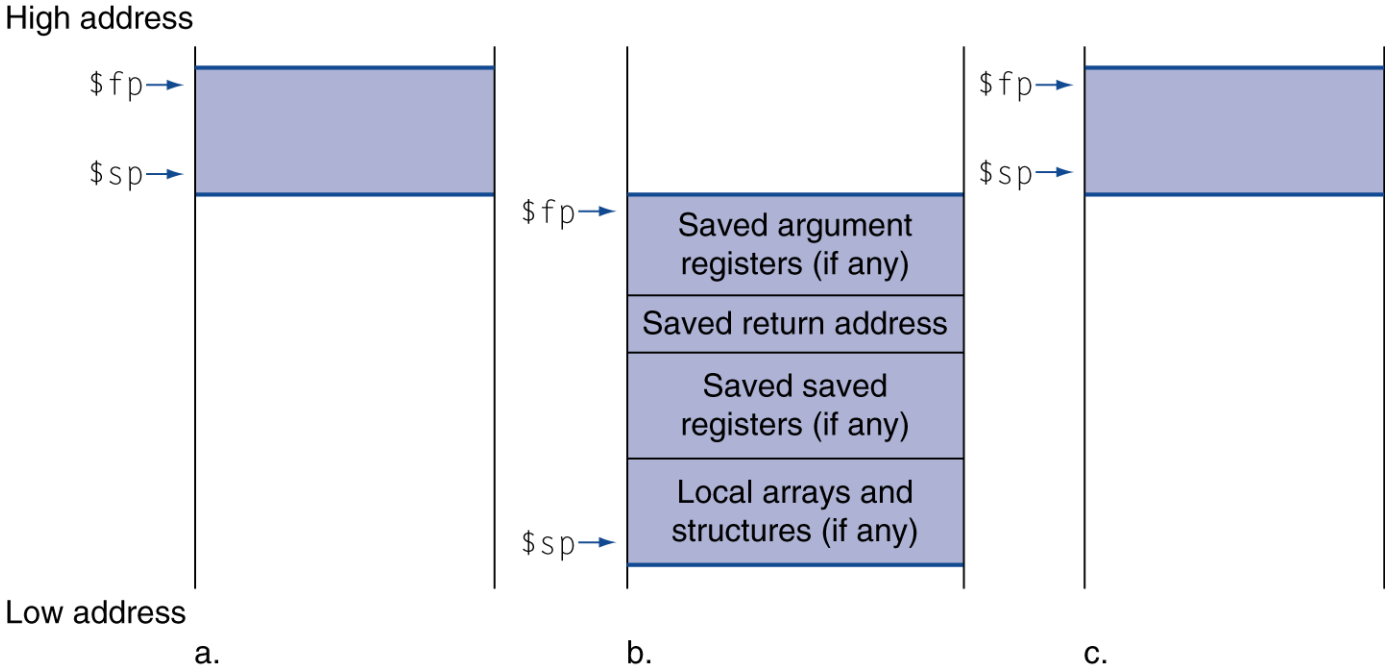
- **$a0 - $a3** : arguments (reg's 4-7)
- **$v0, $v1** : result values (reg's 2 and 3)
- **$t0 - $t9** : temporaries (NOT preserved)
- **$s0 - $s7** : saved (saved/restored by callee)
- **$gp** : global pointer for static data (reg 28)
- **$sp** : stack pointer (reg 29)
- **$fp** : frame pointer (reg 30)
- **$ra** : return address (reg 31)

Stack

- A data structure for spilling registers organized as a **last-in-first-out** (LIFO) queue
- **Stack Pointer** ($sp) : A register for most recently allocated address in a stack
- Stack grows "down" : from high memory address to low memory address

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Local Data on the Stack



- Local data allocated by callee
  - e.g., C automatic variables
- Procedure frame (activation record)
  - Used by some compilers to manage stack storage

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Leaf Procedure Example

- Example C code:

```
int leaf_example (int g, h, i, j)
{
    int f;
    f = ( g + h ) - ( i + j );
    return f;
}
```

```
leaf_example:
```

A.

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Non-Leaf Procedure

- Procedures that call other procedures
- For nested call, caller needs to save on the stack:
  - Its return address
  - Any arguments and temporaries needed after the call
- Restore from the stack after the call
- Example C code:

```
int fact (int n)
{
    if (n < 1) return f;
    else return n * fact(n - 1);
}
```

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Non-Leaf Procedure Example

- Example C code:

```
int fact (int n)
{
    if (n < 1) return f;
    else return n * fact(n - 1);
}
```

fact:

A.

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Non-Leaf Procedure Example

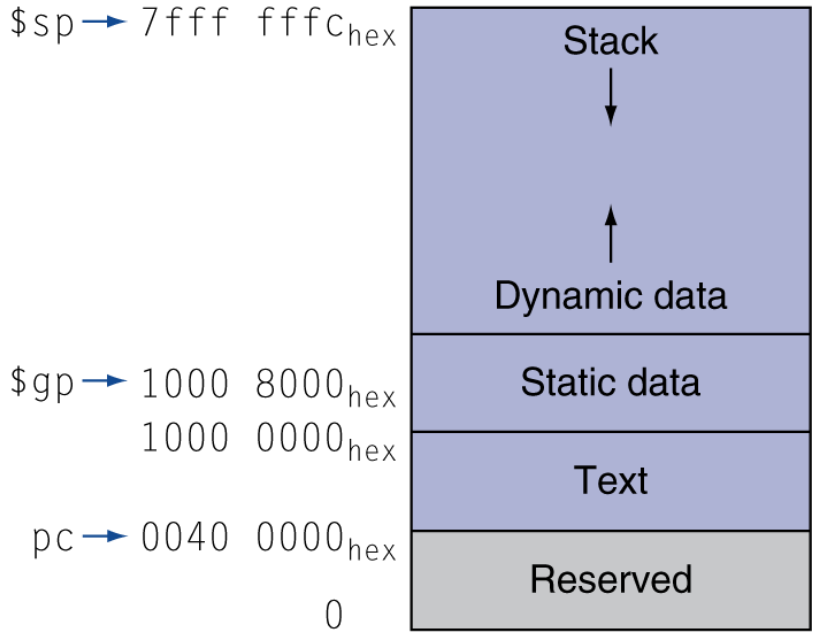
- Example C code:
  - Draw stack when n=5

A.

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Memory Layout

- **Text**: program code
- **Static data**: global variables
  - e.g., static variables in C, constant arrays and strings
  - $gp initialized to address allowing ±offsets into this segment
- **Dynamic data**: heap
  - e.g., malloc/free in C, new/gc in Java
- **Stack**: automatic storage



Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Byte/Halfword Operations

- MIPS byte/halfword load/store
  - String processing is a common case

- Sign extend to 32 bits in rt

```
lb rt, offset(rs) / lh rt, offset(rs)
```

- Zero extend to 32 bits in rt

```
lbu rt, offset(rs) / lhu rt, offset(rs)
```

- Store just rightmost byte/halfword

```
sb rt, offset(rs) / sh rt, offset(rs)
```

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

String Copy

- Example C code:

```
void strcpy (char x[], char y[])
{
    int i; i=0;
    while ((x[i]=y[i]) != '\0') i += 1;
}
```

- Addresses of x, y in $a0, $a1
- i in $s0

```
strcpy:
```

A.

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

32-bit Constants

- Most constants are small
  - 16-bit immediate is sufficient : I-format

- For occasional 32-bit constant

```
lui rt, constant
```

  - load-upper-immediate: Copies 16-bit constant to left(upper) 16 bits of rt
  - Clears right 16 bits of rt to 0
  - Use `ori` to set the right (lower) 16 bits of a register (don't use addi)

```
lui $s0, 61
```

0000 0000 0111 1101	0000 0000 0000 0000

```
ori $s0, $s0, 2304
```

0000 0000 0111 1101	0000 1001 0000 0000

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

MIPS Instruction Set

- Stanford MIPS commercialized by MIPS technologies (www.mips.com)
- Large share of embedded core market
- Typical of many modern ISAs

Classification

Classification by types

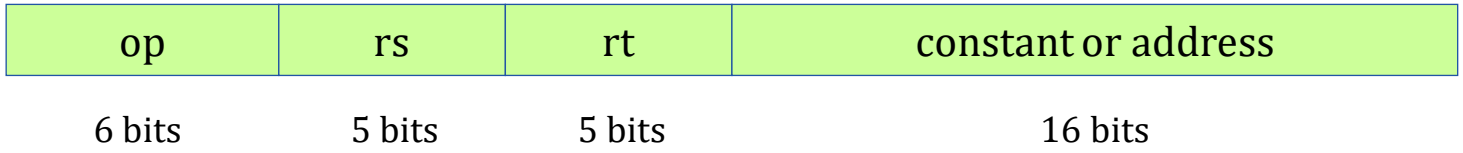
- Arithmetic Instructions
- Memory (Data Transfer) Instructions
- Logical Instructions
- Conditional Instructions
- Branch/Jump Instructions
- Floating Point Instructions
- Pseudo Instructions

Classification by formats

- R-format
- I-format
- J-format
- Floating Point Instructions format

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

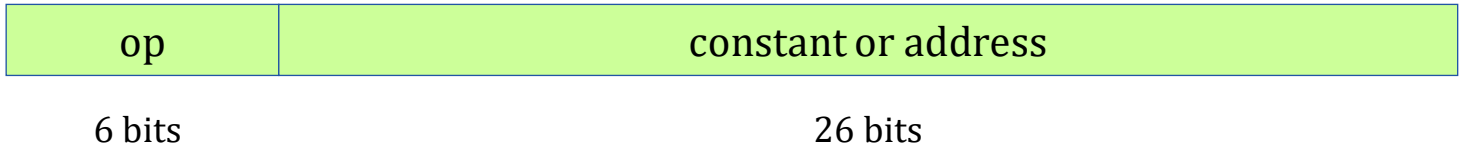
Branch Addressing



- Branch Instructions (e.g., `beq`, `bne`) specify
  - Opcode, two registers, target address : I-format
- Most branch targets are near branch
  - Near PC (current instruction)
  - e.g., loops and if statements
- "PC-relative addressing"
  - Target address = (PC+4) + offset x 4
  - Can branch within $\pm 2^{15}$ words (= $\pm 2^{15}$ instructions = $\pm 2^{17}$ bytes)

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Jump Addressing



- Jump (`j` and `jal`) targets could be anywhere in text segment
  - Far away from PC
  - Encode full (absolute) address in instruction : J-format
- (Pseudo) "Direct jump addressing"
  - Target address = address x 4
  - Can jump to any address within 2^{26} words (= 2^{28} bytes)
- Jump register (`jr`)
  - Full 32-bit address in the register. R-format
- If branch target is too far to encode with 16-bit offset, assembler rewrites the code

```
beq $s0, $s1, L1
```

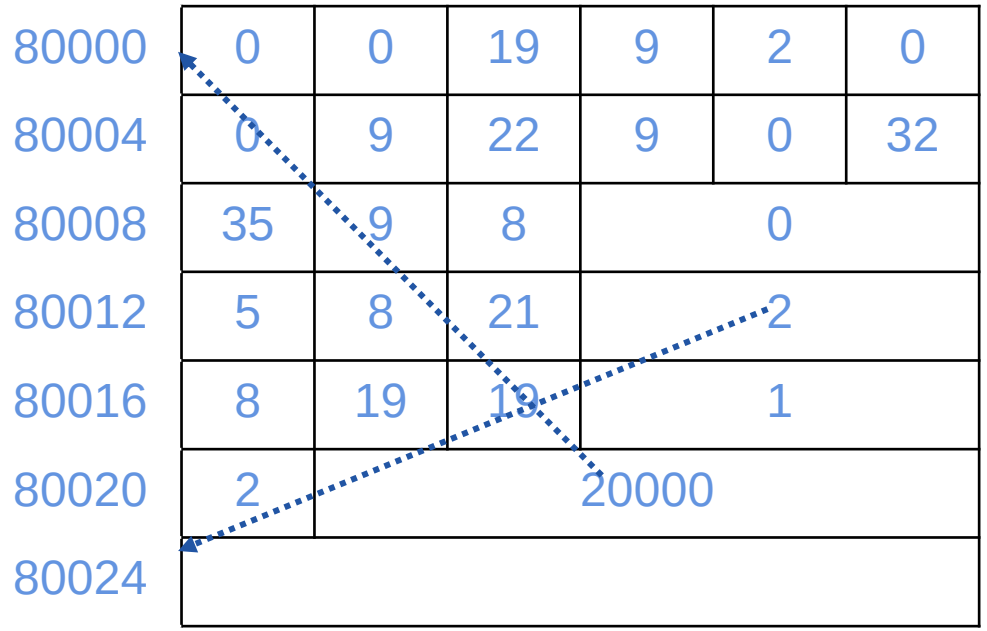
```
beq $s0, $s1, L2
L2: j L1
```

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Target Addressing Example

```
while (save[i] == k)
    i += 1;
```

```
Loop: sll  $t1, $s3, 2
      add  $t1, $t1, $s6
      lw   $t0, 0($t1)
      bne  $t0, $s5, Exit
      addi $s3, $s3, 1
      j    Loop
Exit: …
```



Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Addressing Mode Summary

1. Immediate addressing

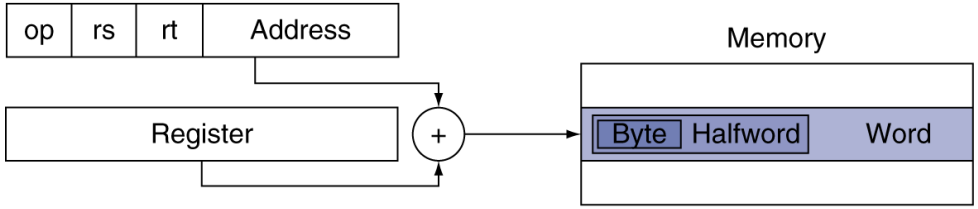


2. Register addressing



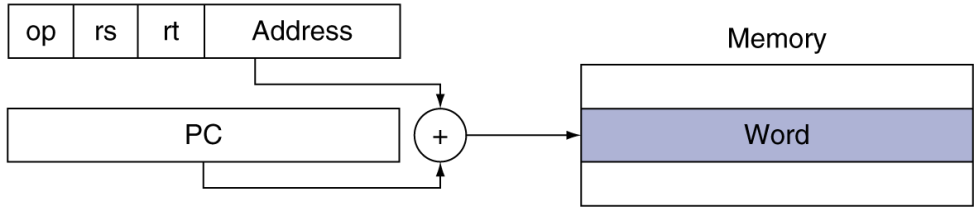
ex) **JR**

3. Base addressing



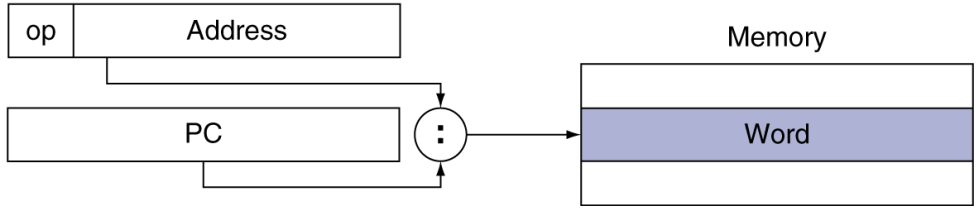
ex) **LW, SW**

4. PC-relative addressing



ex) **BEQ, BNE**

5. Pseudodirect addressing



ex) **J, JAL**

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

MIPS Instruction Set

- Stanford MIPS commercialized by MIPS technologies (www.mips.com)
- Large share of embedded core market
- Typical of many modern ISAs

Classification

Classification by types

- Arithmetic Instructions
- Memory (Data Transfer) Instructions
- Logical Instructions
- Conditional Instructions
- Branch/Jump Instructions
- Floating Point Instructions
- Pseudo Instructions

Classification by formats

- R-format
- I-format
- J-format
- Floating Point Instructions format

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

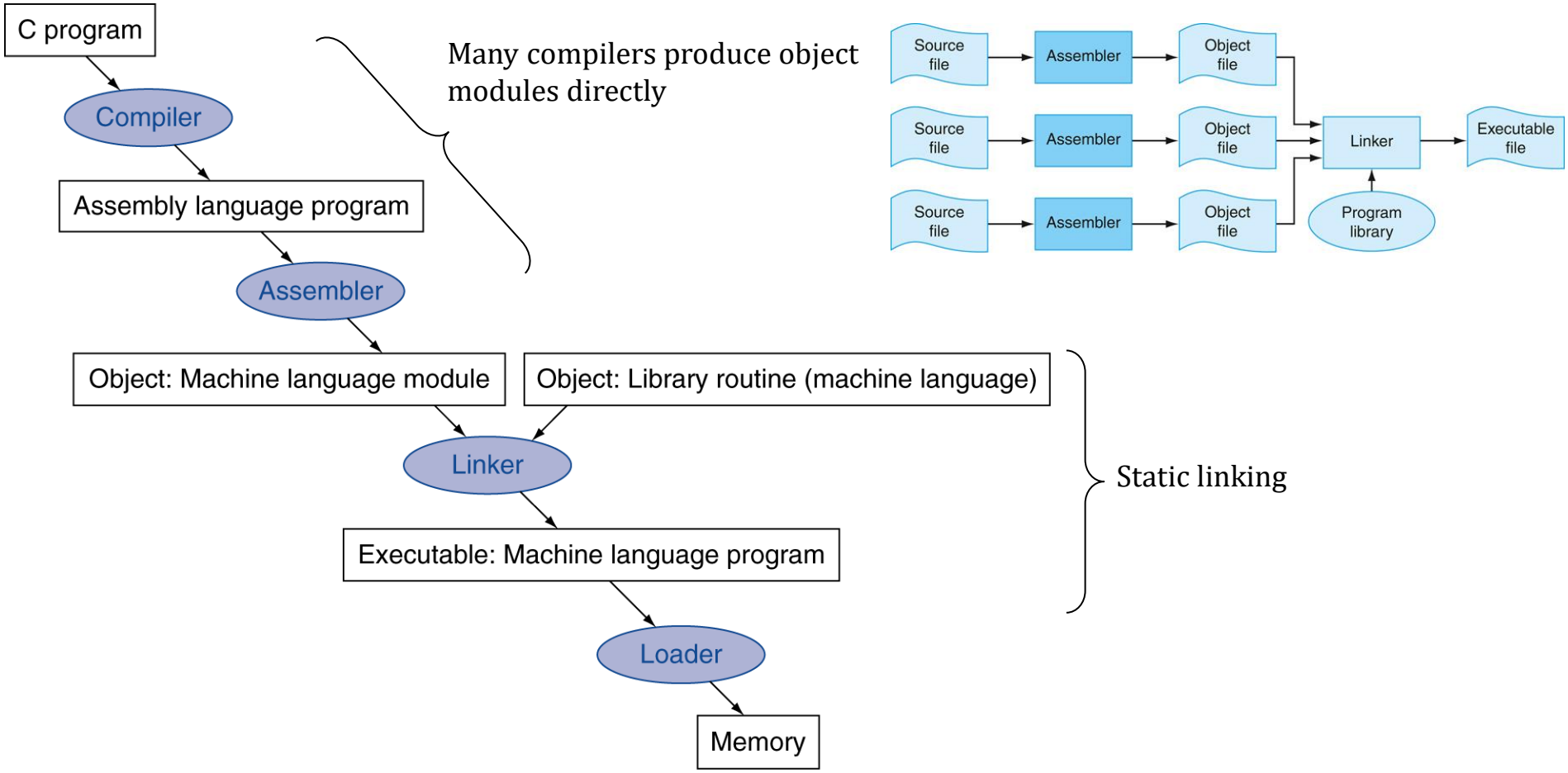
Assembler Pseudoinstructions

- Most assembler instructions represent machine instructions one-to-one
- Pseudo-instructions: figments of the assembler's imagination

```
move $t0, $t1          →    add $t0, $zero, $t1
blt $t0, $t1, L        →    slt $at, $t0, $t1
                            bne $at, $zero, L
```

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Translation and Startup



Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Producing an Object Module

- Assembler (or compiler) translates program into machine instructions
- Provides information for building a complete program from the pieces
  - Header : described contents of object module
  - Text segment : translated instructions
  - Static data segment : data allocated for the life of the program
  - Relocation information : for contents that depend on absolute location of loaded program
  - Symbol table : global definitions and external references
  - Debug information : for associating with source code

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Linking Object Modules

- Produces an executable image
  1. Merges Segments
  2. Resolve labels (determine their addresses)
  3. Patch location-dependent and external refs

- Could leave location dependencies for fixing by a relocating loader
  - But with virtual memory, no need to do this
  - Program can be loaded into absolute location in virtual memory space

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Loading a Program

- Load from image file on disk into memory

1. Read header to determine segment sizes
2. Create virtual address space
3. Copy text and initialized data into memory
4. Set up arguments on stack
5. Initialize registers (including $sp, $fp, $gp)
6. Jump to startup routine (Copies argument and calls main, main returns do exit syscall)

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Dynamic Linking

- Only link/load library procedure when it is called
  - Requires procedure code to be relocatable
  - Avoids image bloat caused by static linking of all (transitively) referenced libraries
  - Automatically picks up new library versions

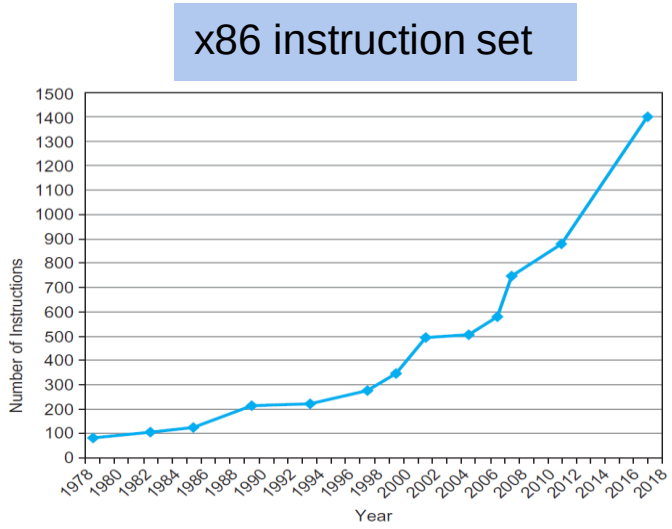
Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Fallacies

- Powerful instruction = Higher Performance
  - Fewer instructions required
  - Complex instructions are hard to implement (May slow down all instructions including simple ones)
  - Compilers are good at making fast code from simple instructions

- Use assembly code for high performance
  - Modern compilers are better at dealing with modern processors
  - More lines of code = more errors and less productivity

- Backward Compatibility
  - Instruction Set doesn't Change
  - But they do accrete more instructions



Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Concluding Remarks

- Design principles
  1. Simplicity favors Regularity
  2. Smaller is faster
  3. Make the common case fast
  4. Good design demands Good Compromises

- MIPS : typical of RISC ISAs

  c.f. x86

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Concluding Remarks

- Measuring MIPS instruction executions in benchmark programs
  - Consider making the common case fast
  - Consider compromises

Instruction class	MIPS examples	SPEC2006 Int	SPEC2006 FP
Arithmetic	`add, sub, addi`	16%	48%
Data transfer	`lw, sw, lb, lbu, lh, lhu, sb, lui`	35%	36%
Logical	`and, or, nor, andi, ori, sll, srl`	12%	4%
Cond. Branch	`beq, bne, slt, slti, sltiu`	34%	8%
Jump	`j, jr, jal`	2%	0%

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential



Thank you

Bumsoo Kim

School of Computer Science & Engineering

E-mail : bumsoo@cau.ac.kr

Office : 310-430